

# Guia de Arquitetura OCI - Hackathon PodAcademy 2025

## Objetivo deste Documento

Este guia foi criado para ajudar os integrantes do grupo a:

- Entender a arquitetura completa do projeto
- Preparar-se para apresentações
- Responder perguntas técnicas sobre o desenho

**Complementar a:** `OCI_OPERACIONAL.md` (detalhes técnicos de implementação)

## Índice

1. [Visão Geral da Arquitetura](#)
2. [Componentes Externos \(Fora da OCI\)](#)
3. [Estrutura de Rede \(VCN\)](#)
4. [Gateways \(Portões de Entrada/Saída\)](#)
5. [Componentes de Processamento](#)
6. [Armazenamento \(Object Storage\)](#)
7. [Orquestração \(Airflow\)](#)
8. [Segurança e Governança](#)
9. [Fluxos Detalhados](#)
10. [Perguntas Frequentes \(FAQ\)](#)
11. [Glossário](#)
12. [Checklist de Preparação](#)
13. [Recursos Adicionais](#)
14. [Resumo Executivo](#)

# 1. Visão Geral da Arquitetura

## 1.1 O que estamos construindo?

Um sistema de **scoring de crédito** para a Claro que:

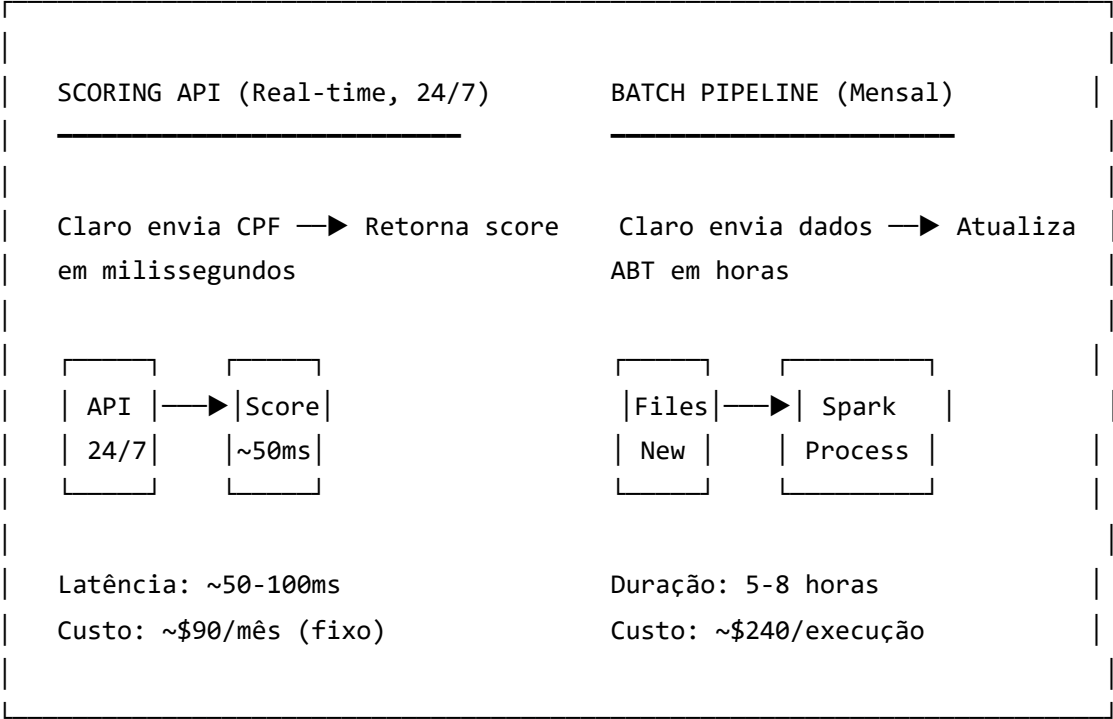
- 1. **Processa dados históricos** (batch) para treinar um modelo de Machine Learning
- 2. **Responde em tempo real** se um cliente deve ser aprovado ou não (scoring)

## 1.2 Dois Fluxos Principais

| Fluxo             | Tipo      | Frequência         | Disponibilidade | Objetivo                        |
|-------------------|-----------|--------------------|-----------------|---------------------------------|
| Scoring API       | Real-time | A cada solicitação | 24/7            | Retornar decisão de crédito     |
| Pipeline de Dados | Batch     | Mensal             | Sob demanda     | Processar dados e atualizar ABT |

**Importante:** A Claro pode chamar a API a qualquer momento. O pipeline batch roda apenas quando a Claro entrega novos dados (mensalmente).

## 1.3 Diferença entre os Cenários



## 1.4 Diagrama de Referência

O diagrama está disponível em: [docs/architecture/diagrams/07.png](#)

## 2. Componentes Externos (Fora da OCI)

### 2.1 System Claro (Platform)

O que é:

- Sistema/plataforma interna da Claro
- É quem consome nossa API de scoring

Função:

- Quando um cliente solicita migração/crédito, a plataforma da Claro chama nossa API
- Recebe a resposta (aprovar/reprovar) e informa o cliente

**Pergunta comum:** *"O cliente final acessa diretamente a API?"*

**Resposta:** Não. O cliente interage com a plataforma da Claro. A plataforma da Claro é quem chama nossa Scoring API internamente.

### 2.2 Database System

O que é:

- Banco de dados da Claro com dados históricos dos clientes
- Contém: bureau, telco, cadastro, recarga, pagamento, atraso

Função:

- Fonte de dados para o pipeline de ETL
- Os dados são extraídos periodicamente para alimentar o modelo

**Pergunta comum:** *"Quem extrai os dados do banco?"*

**Resposta:** O serviço **Data Integration** da OCI faz a extração e envia para o bucket Landing.

# 3. Estrutura de Rede (VCN)

## 3.1 O que é VCN?

**VCN (Virtual Cloud Network)** é a rede virtual privada dentro da OCI. É como criar sua própria rede isolada na nuvem.

| Propriedade | Valor                     |
|-------------|---------------------------|
| Nome        | VCN SQUAD_8               |
| CIDR        | 10.0.0.0/16               |
| Região      | sa-saopaulo-1 (São Paulo) |

**Pergunta comum:** "O que significa 10.0.0.0/16?"

**Resposta:** É a faixa de IPs disponíveis na rede. O "/16" significa que temos 65.536 endereços IP disponíveis (de 10.0.0.0 até 10.0.255.255).

## 3.2 Subnets (Sub-redes)

A VCN é dividida em **3 subnets**, cada uma com um propósito específico:

### Public Subnet (10.0.1.0/24)

| Propriedade | Valor                                    |
|-------------|------------------------------------------|
| Tipo        | <b>Pública</b> (acessível pela internet) |
| CIDR        | 10.0.1.0/24 (256 IPs)                    |
| Componentes | Load Balancer, NAT Gateway               |

**Por que é pública?**

- O Load Balancer precisa receber requisições da internet
- É a "porta de entrada" da nossa aplicação

## Private Subnet Compute (10.0.5.0/24)

| Propriedade | Valor                                        |
|-------------|----------------------------------------------|
| Tipo        | <b>Privada</b> (não acessível pela internet) |
| CIDR        | 10.0.5.0/24 (256 IPs)                        |
| Componentes | Scoring API                                  |

### Por que é privada?

- A API contém lógica de negócio sensível
- Não deve ser acessada diretamente pela internet
- Só recebe tráfego do Load Balancer

## Private Subnet Data (10.0.3.0/24)

| Propriedade | Valor                   |
|-------------|-------------------------|
| Tipo        | <b>Privada</b>          |
| CIDR        | 10.0.3.0/24 (256 IPs)   |
| Componentes | Data Flow, Data Science |

### Por que é privada?

- Processa dados sensíveis dos clientes
- Não precisa de acesso externo direto

### Pergunta comum: "Por que separar em várias subnets?"

#### Resposta:

1. **Segurança:** Isolar componentes com diferentes níveis de exposição
2. **Controle:** Aplicar regras de firewall (Security Lists) específicas
3. **Organização:** Facilitar gerenciamento e troubleshooting

## 4. Gateways (Portões de Entrada/Saída)

### 4.1 Internet Gateway

O que é:

- Portão que permite comunicação entre a VCN e a internet

Função:

- Permite que o Load Balancer receba requisições externas
- Sem ele, nada de fora consegue entrar na VCN

**Analogia:** É a porta da frente do prédio.

### 4.2 NAT Gateway

O que é:

- Network Address Translation Gateway

Função:

- Permite que recursos em subnets **privadas** acessem a internet
- Mas **não permite** que a internet acesse esses recursos

Exemplo de uso:

- Data Flow precisa baixar bibliotecas Python (pip install)
- Data Science precisa acessar repositórios externos

**Analogia:** É como um porteiro que deixa você sair, mas não deixa estranhos entrarem.

**Pergunta comum:** *"Por que não colocar tudo na subnet pública?"*

**Resposta:** Segurança. Recursos que não precisam ser acessados externamente devem ficar em subnets privadas. O NAT Gateway permite que eles acessem a internet quando necessário, sem ficarem expostos.

## 4.3 Service Gateway

O que é:

- Conexão privada entre a VCN e serviços da OCI (como Object Storage)

Função:

- Permite que Data Flow e Scoring API acessem o Object Storage
- O tráfego **não passa pela internet** - fica dentro da rede da OCI

Vantagens:

| Aspecto    | Com Service Gateway | Sem Service Gateway |
|------------|---------------------|---------------------|
| Velocidade | Mais rápido         | Mais lento          |
| Custo      | Gratuito            | Paga transferência  |
| Segurança  | Rede interna        | Passa pela internet |

**Analogia:** É um corredor interno do prédio que conecta salas sem precisar sair na rua.

## 5. Componentes de Processamento

### 5.1 Load Balancer

O que é:

- Balanceador de carga gerenciado pela OCI

Função:

- Recebe requisições da internet (Sistema Claro)
- Distribui para a Scoring API
- Faz health check (verifica se a API está funcionando)

Por que usar:

- Ponto de entrada seguro e controlado
- Pode ter múltiplas instâncias da API (escalabilidade)

- SSL/TLS termination (HTTPS)

## 5.2 Scoring API (24/7)

### O que é:

- API REST que executa o modelo de Machine Learning
- **Roda 24 horas por dia, 7 dias por semana**
- Hospedada em uma VM dedicada (sempre ligada)

### Por que precisa estar sempre ligada?

- A Claro pode solicitar um score a qualquer momento
- Latência deve ser baixa (~50-100ms)
- Não podemos ter "cold start" (tempo de inicialização)

### Infraestrutura:

| Componente | Especificação                 |
|------------|-------------------------------|
| VM Shape   | VM.Standard.E4.Flex           |
| CPU        | 1 OCPU                        |
| Memória    | 8 GB RAM                      |
| Framework  | FastAPI (Python)              |
| Modelo     | LightGBM carregado em memória |
| Custo      | ~\$35/mês                     |

### O que roda na VM:



```
VM Scoring API
├─ Python 3.9+
├─ FastAPI (framework web)
├─ Uvicorn (servidor ASGI)
├─ LightGBM (modelo ~50MB em memória)
├─ Systemd service (auto-restart se cair)
└─ /app/
    ├── main.py          # Código da API
    ├── models/
    │   └─ modelo_fpd.txt # Modelo treinado
    └─ requirements.txt
```

**Exemplo de request/response:**

```
// Request
POST /v1/score
{
  "cpf": "12345678900",
  "score_01": 650,
  "score_02": 720,
  "freq_sos_m1": 2.0,
  "ticket_medio_m1": 25.50
}

// Response
{
  "cpf": "12345678900",
  "score": 850,
  "probability": 0.15,
  "risk_class": "LOW",
  "decision": "APPROVED",
  "timestamp": "2026-02-09T10:30:00Z"
}
```

**Endpoints da API:**

| Endpoint  | Método | Função                           |
|-----------|--------|----------------------------------|
| /v1/score | POST   | Calcula score do cliente         |
| /health   | GET    | Health check (Load Balancer usa) |

| Endpoint | Método | Função                      |
|----------|--------|-----------------------------|
| /metrics | GET    | Métricas para monitoramento |

**Pergunta comum:** "O modelo fica na API?"

**Resposta:** O modelo fica armazenado no bucket **Models**. A API carrega o modelo **uma vez** na inicialização e mantém em memória. Não recarrega a cada requisição.

**Pergunta comum:** "O que acontece se a VM cair?"

**Resposta:** O systemd service reinicia automaticamente. O Load Balancer detecta via health check e para de enviar tráfego até a API estar saudável novamente.

## 5.3 Data Flow (Spark) - Execução Mensal

O que é:

- Serviço gerenciado de Apache Spark da OCI
- **Roda sob demanda** (não fica ligado 24/7)

Quando executa:

- **Mensalmente**, quando a Claro entrega novos dados no bucket Landing
- Pode ser disparado por evento (arquivo chegou) ou agendamento

Função:

- Executa os jobs de ETL (Extract, Transform, Load)
- Processa grandes volumes de dados (95M+ registros)
- Transforma dados: Landing → Bronze → Silver → Gold

Jobs do Pipeline:

| Job              | Descrição                                  | Duração | Custo  |
|------------------|--------------------------------------------|---------|--------|
| Bronze Ingestion | Landing → Bronze                           | ~30 min | ~\$15  |
| Silver Transform | Bronze → Silver                            | ~1-2h   | ~\$60  |
| Gold Features    | Silver → Recarga/Pagamento/Atraso features | ~2-3h   | ~\$120 |

| Job          | Descrição                  | Duração      | Custo         |
|--------------|----------------------------|--------------|---------------|
| ABT Builder  | Monta ABT v6 (614 colunas) | ~1-2h        | ~\$45         |
| <b>Total</b> |                            | <b>~5-8h</b> | <b>~\$240</b> |

### Por que Spark:

- Processamento distribuído (paralelo)
- Escala automaticamente conforme volume de dados
- Suporte a Delta Lake

### Pergunta comum: "Por que não usar Python puro?"

**Resposta:** Volume de dados. Com 95 milhões de registros de recarga, Python puro demoraria horas/dias. Spark distribui o processamento em múltiplos nós e termina em minutos/horas.

### Pergunta comum: "O Data Flow fica ligado o tempo todo?"

**Resposta:** Não! É **sob demanda**. Liga quando precisa processar dados, desliga quando termina. Por isso o custo é ~240/ execução não 240/mês.

## 5.4 Data Science (ML Studio)

### O que é:

- Serviço de Machine Learning da OCI (notebooks Jupyter gerenciados)

### Função:

- Análise exploratória dos dados
- Seleção de features (variáveis)
- Treinamento do modelo LightGBM
- Avaliação de métricas (KS, AUC, Gini)
- Exportação do modelo para o bucket Models

### Fluxo:

Gold bucket → Data Science → Treina modelo → Models bucket

# 6. Armazenamento (Object Storage)

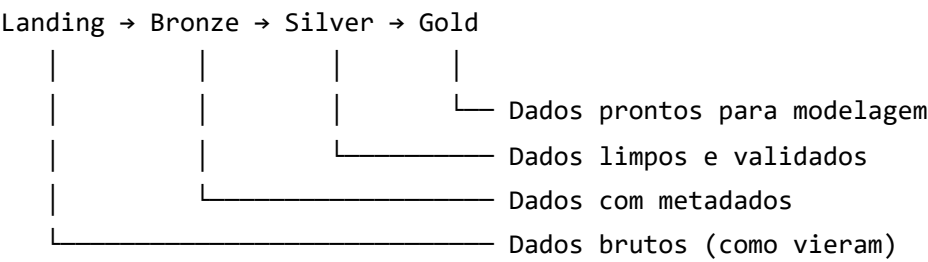
## 6.1 O que é Object Storage?

- Armazenamento de objetos (arquivos) da OCI
- Altamente durável e escalável
- **Fica fora da VCN** - é um serviço regional

## 6.2 Buckets do Projeto

| Bucket  | Conteúdo                 | Escrito por      | Lido por     |
|---------|--------------------------|------------------|--------------|
| Landing | Dados brutos (Parquet)   | Data Integration | Data Flow    |
| Bronze  | Dados + metadados        | Data Flow        | Data Flow    |
| Silver  | Dados tipados, validados | Data Flow        | Data Flow    |
| Gold    | ABT (features agregadas) | Data Flow        | Data Science |
| Models  | Modelo treinado (.txt)   | Data Science     | Scoring API  |

## 6.3 Medallion Architecture



**Pergunta comum:** *"Por que não processar direto do Landing para o Gold?"*

**Resposta:**

1. **Rastreabilidade:** Se algo der errado, sabemos em qual etapa foi
2. **Reprocessamento:** Podemos reprocessar a partir de qualquer camada
3. **Qualidade:** Cada camada adiciona validações

## 7. Orquestração (Airflow)

### 7.1 O que é Airflow?

- Orquestrador de workflows (fluxos de trabalho)
- Agenda e coordena a execução de jobs

### 7.2 Função no Projeto

#### Sem Airflow:

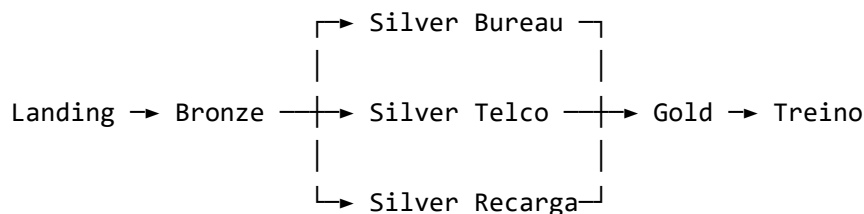
```
Engenheiro acorda 6h da manhã
→ Roda job Bronze manualmente
→ Espera terminar
→ Roda job Silver manualmente
→ Espera terminar
→ ...
```

#### Com Airflow:

```
Airflow agenda para 6h:
→ 06:00 - Roda Bronze automaticamente
→ 07:00 - Roda Silver (após Bronze terminar)
→ 08:00 - Roda Gold
→ 09:00 - Roda treino do modelo
→ Notifica equipe se algo falhar
```

### 7.3 DAGs (Directed Acyclic Graphs)

Airflow organiza jobs em DAGs - grafos de dependência:



# 8. Segurança e Governança

## 8.1 Security Lists

O que são:

- Regras de firewall para cada subnet
- Controlam qual tráfego pode entrar/sair

Exemplo - Public Subnet:

| Direção | Protocolo | Porta | Origem    | Permissão         |
|---------|-----------|-------|-----------|-------------------|
| Entrada | TCP       | 443   | 0.0.0.0/0 | HTTPS da internet |
| Entrada | TCP       | 22    | IP Admin  | SSH (restrito)    |
| Saída   | Todos     | Todas | 0.0.0.0/0 | Liberado          |

## 8.2 IAM Groups

Controle de acesso baseado em grupos:

| Grupo         | Permissões                           |
|---------------|--------------------------------------|
| Admin         | Acesso total a todos os recursos     |
| DataEngineer  | Storage, Data Flow, Data Integration |
| DataScientist | Data Science, leitura de Storage     |

## 8.3 Monitoring

O que monitora:

- Uso de CPU/memória dos recursos
- Tempo de resposta da API
- Erros e falhas
- Custos

Alertas configurados:

- API com tempo de resposta > 2 segundos

- Job Data Flow falhou
- Custo diário > limite

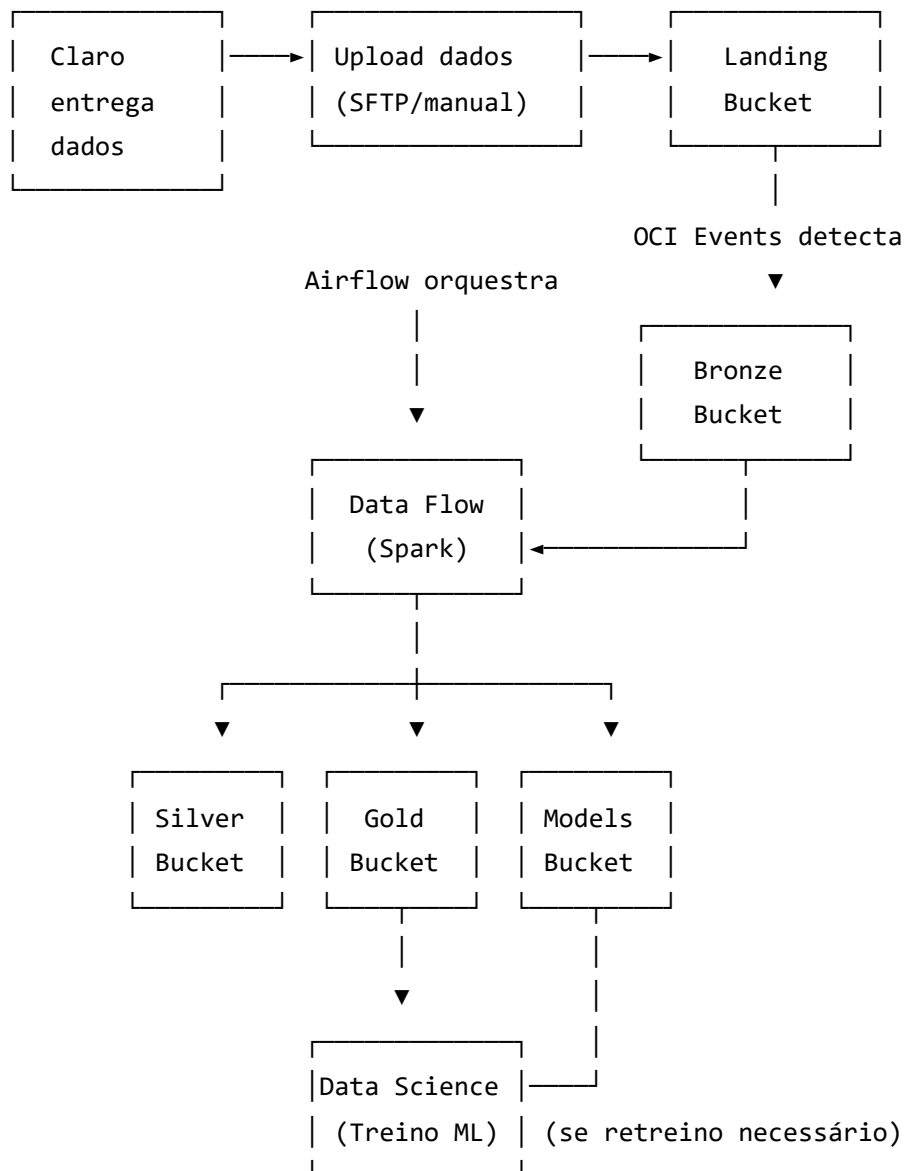
## 9. Fluxos Detalhados

### 9.1 Fluxo Batch (Pipeline de Dados) - MENSAL

**Quando acontece:** Quando a Claro entrega novos dados (geralmente mensal)

**Trigger:**

- **Opção A (Recomendada):** Event-driven - Airflow detecta novos arquivos no Landing
- **Opção B:** Agendado - Airflow roda todo dia 5 do mês



**Frequência:** Mensal (quando Claro entrega novos dados)

**Duração total:** 5-8 horas

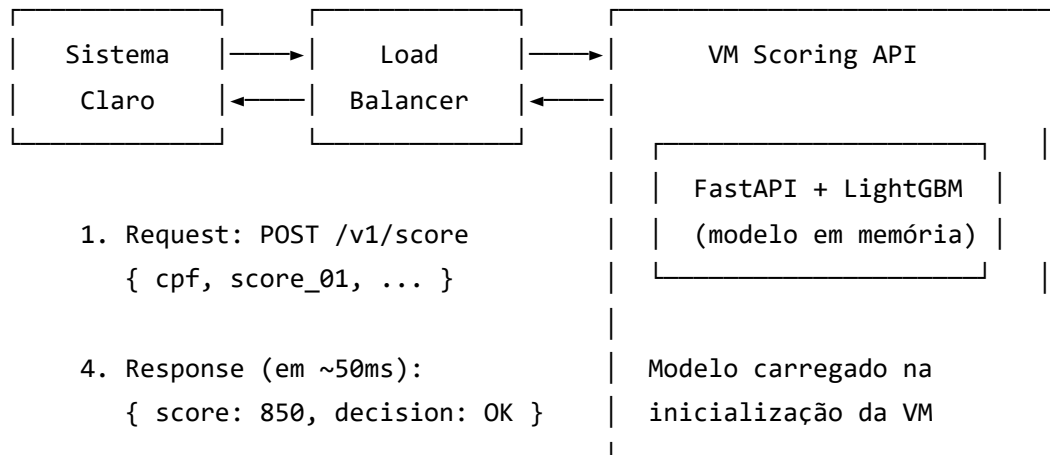
**Custo por execução:** ~\$240

## 9.2 Fluxo Real-time (Scoring) - 24/7

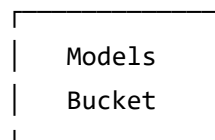
**Disponibilidade:** 24 horas por dia, 7 dias por semana

**Quando acontece:** A qualquer momento que a Claro precisar avaliar um cliente





Modelo foi treinado pelo batch anterior



**Latência esperada:** ~50-100ms por requisição (muito rápido!)

**Por que é rápido?**

- Modelo já está carregado em memória (não lê do bucket a cada request)
- FastAPI é assíncrono e eficiente
- LightGBM é otimizado para inferência

**Custo fixo:** ~\$90/mês (VM + Load Balancer + NAT Gateway)

## 10. Perguntas Frequentes (FAQ)

### Operação 24/7

**P: A VM da Scoring API precisa ficar ligada o tempo todo?**

R: **Sim.** A Claro pode chamar a API a qualquer momento. Uma VM desligada não responde requisições. Por isso usamos uma VM always-on (~\$35/mês).

**P: E se a VM cair no meio da noite?**

R: O systemd service reinicia automaticamente. Se não conseguir, o Load Balancer detecta via health check e podemos configurar alertas para o time.

**P: Por que não usar OCI Functions (serverless) ao invés de VM?**

R: **Cold start.** Functions demoram 1-3 segundos para "acordar" após ficarem ociosas. Para scoring em tempo real, isso é inaceitável. A VM tem latência constante de ~50ms.

**P: Quanto custa manter a API 24/7?**

R:  $\sim 90/mês (VM35 + \text{Load Balancer } 20 + NATGateway35)$ . É um custo fixo independente do número de requisições.

## Pipeline Batch

**P: O pipeline roda todo dia?**

R: **Não.** Roda mensalmente, quando a Claro entrega novos dados. Entre entregas, não há processamento (e não há custo de Data Flow).

**P: Como sabemos que novos dados chegaram?**

R: OCI Events monitora o bucket Landing. Quando um arquivo novo é criado, dispara o Airflow automaticamente.

**P: Quanto tempo leva o pipeline completo?**

R: 5-8 horas para processar todos os dados (Landing → Bronze → Silver → Gold → ABT).

**P: Quanto custa cada execução do pipeline?**

R: ~\$240 por execução mensal (Data Flow sob demanda).

## Rede e Conectividade

**P: Por que usar subnets privadas se complica o acesso?**

R: Segurança. Dados sensíveis de clientes não devem ficar expostos à internet. O custo de complexidade é menor que o risco de vazamento.

**P: O que acontece se o NAT Gateway cair?**

R: Recursos privados perdem acesso à internet, mas o sistema principal (Scoring API) continua funcionando pois usa Service Gateway para acessar o Storage.

**P: Por que não usar VPN ao invés de Load Balancer?**

R: VPN é para acesso administrativo. O Load Balancer é para tráfego de aplicação em escala, com balanceamento e health checks.

## Processamento

**P: Por que Spark e não Python puro?**

R: Volume. 95 milhões de registros. Spark processa em paralelo, Python seria sequencial e muito lento.

**P: O modelo é retreinado a cada requisição?**

R: **Não.** O modelo é treinado no batch (mensal) e a API apenas faz inferência (predição) usando o modelo já treinado em memória.

## Custos

**P: Quanto custa essa arquitetura em produção?**

R: **~\$350/mês** em operação normal:

- Scoring API 24/7: ~\$90/mês (fixo)
- Pipeline Batch: ~\$240/mês (1 execução mensal)
- Storage + Monitoring: ~\$20/mês

**P: Por que o documento antigo falava em \$1.000-1.500?**

R: Aquele era o custo de **desenvolvimento** (3 execuções de Data Flow, notebooks de experimentação). Em produção, rodamos 1x por mês apenas.

**P: Como otimizar custos?**

R:

- Usar VM menor se throughput permitir (0.5 OCPU)
- Não retreinar modelo todo mês se KS estiver estável
- Mover dados antigos para Archive Storage

## Segurança

**P: Os dados dos clientes estão seguros?**

R: Sim. Dados ficam em subnets privadas, criptografados em repouso (SSE), e acessados apenas via Service Gateway (rede interna OCI).

**P: Quem pode acessar o quê?**

R: Controlado por IAM Groups. DataEngineers não acessam produção de API, DataScientists não alteram pipelines, etc.

## Atualização do Modelo

**P: Como atualizo o modelo em produção?**

R:

1. Data Scientist treina novo modelo
2. Valida KS  $\geq$  33% no OOT
3. Upload para bucket Models
4. Restart da Scoring API (carrega novo modelo)
5. Verifica health check

**P: Precisa desligar a API para atualizar o modelo?**

R: Apenas restart rápido (~10 segundos). O Load Balancer detecta e para de enviar tráfego durante o restart.

## 11. Glossário

| Termo         | Definição                                                    |
|---------------|--------------------------------------------------------------|
| <b>VCN</b>    | Virtual Cloud Network - rede virtual privada na OCI          |
| <b>Subnet</b> | Sub-rede dentro da VCN                                       |
| <b>CIDR</b>   | Notação para faixa de IPs (ex: 10.0.0.0/16)                  |
| <b>NAT</b>    | Network Address Translation                                  |
| <b>ETL</b>    | Extract, Transform, Load - processo de movimentação de dados |
| <b>Bucket</b> | Container de armazenamento no Object Storage                 |

| Termo | Definição                                                   |
|-------|-------------------------------------------------------------|
| DAG   | Directed Acyclic Graph - grafo de dependências do Airflow   |
| FPD   | First Payment Default - inadimplência no primeiro pagamento |
| KS    | Kolmogorov-Smirnov - métrica de performance do modelo       |
| ABT   | Analytical Base Table - tabela final para modelagem         |

## 12. Checklist de Preparação para Apresentação

### Antes da apresentação:

- ☐ Revisei este documento completo
- ☐ Entendi os dois fluxos (Batch mensal e Real-time 24/7)
- ☐ Sei explicar a função de cada componente
- ☐ Sei a diferença entre custo de desenvolvimento vs produção
- ☐ Pratiquei responder as perguntas do FAQ
- ☐ Sei os números chave (95M registros, 614 colunas, KS 33.94%)

### Durante a apresentação:

- ☐ Começar pelo objetivo do sistema (scoring de crédito)
- ☐ Explicar os dois fluxos principais (API 24/7 vs Batch mensal)
- ☐ Enfatizar: API sempre ligada, Batch sob demanda
- ☐ Detalhar componentes conforme perguntas surgem
- ☐ Usar analogias quando necessário (porteiro, corredor interno, etc.)

### Números importantes para lembrar:

| Métrica               | Valor           |
|-----------------------|-----------------|
| Registros processados | 95M (Recarga)   |
| Colunas na ABT final  | 614             |
| Features selecionadas | 264 (IV > 0.01) |

| Métrica               | Valor             |
|-----------------------|-------------------|
| KS do modelo (OOT)    | <b>33.94%</b>     |
| Benchmark             | 33.10%            |
| Ganho sobre benchmark | <b>+0.84 p.p.</b> |
| Latência da API       | ~50-100ms         |
| <b>Custo produção</b> | <b>~\$350/mês</b> |
| Custo desenvolvimento | ~\$1.000/mês      |

Custos detalhados (para perguntas):

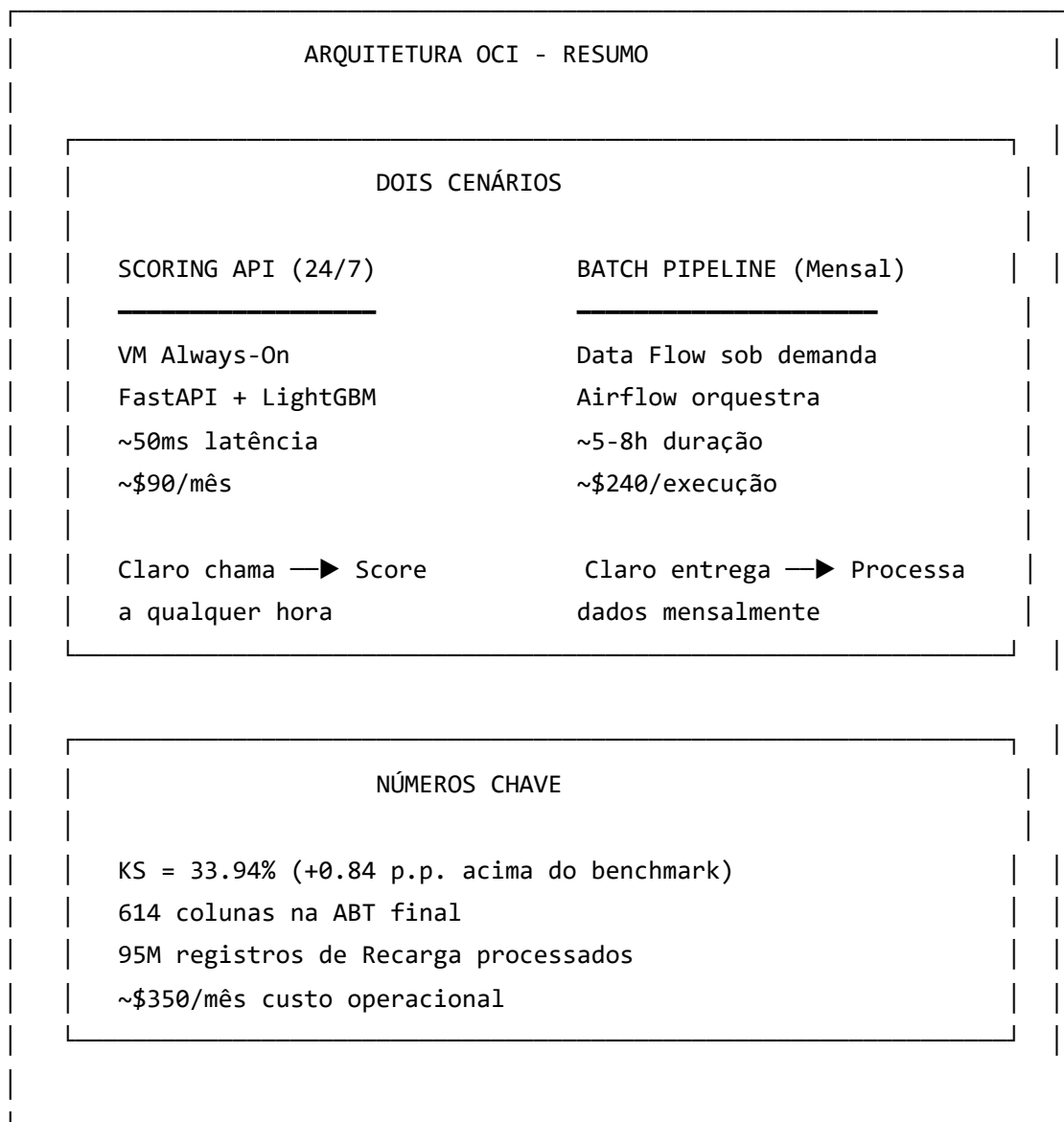
| Componente            | Custo/Mês     | Modo             |
|-----------------------|---------------|------------------|
| VM Scoring API        | \$35          | Always-on (24/7) |
| Load Balancer         | \$20          | Always-on        |
| NAT Gateway           | \$35          | Always-on        |
| Data Flow             | \$240         | 1x mensal        |
| Storage               | \$7.50        | 295 GB           |
| Monitoring            | \$10          | Basic            |
| <b>Total Produção</b> | <b>~\$350</b> |                  |

13. Recursos Adicionais

| Documento                    | Caminho                               | Descrição                                       |
|------------------------------|---------------------------------------|-------------------------------------------------|
| Arquitetura OCI Base         | docs/architecture/OCI_ARCHITECTURE.md | Detalhes técnicos de cada recurso               |
| <b>Cenários Operacionais</b> | docs/architecture/OCI_OPERACIONAL.md  | <b>API 24/7, Batch mensal, deploy do modelo</b> |

| Documento           | Caminho                                     | Descrição                 |
|---------------------|---------------------------------------------|---------------------------|
| Terraform + Airflow | docs/architecture/OCI_TERRAFORM_AIRFLOW.md  | Código de infraestrutura  |
| Diagrama            | docs/architecture/diagrams/07.png           | Versão final do diagrama  |
| Variable Book       | docs/04_gold_rules/BOOK_VARIABLES_ABT_V6.md | Dicionário de variáveis   |
| Target Definition   | docs/00_project/target_definition.md        | Definição do FPD e regras |

## 14. Resumo Executivo (Para Lembrar Rápido)



*Documento criado em: Fevereiro 2026*

*Última atualização: 09/02/2026*

*Projeto: Hackathon PodAcademy 2025 - Modelo de Risco de Crédito*